

Basket

MATTHEWS Louis-Marie, OUNES Sarah

January 5, 2026

Contents

1	Exploring and preparing the dataset	1
1.1	Basic EDA	1
1.2	Preparing the data	2
2	Importing the data into Neo4j Desktop	3
2.1	Creating a bi-partite graph	3
3	Exploring the data with Neo4j	5
3.1	Creating a projection	5
3.1.1	Creating a projection for a monopartite graph	6
4	Analysis	7
4.1	Similarity Analysis	7
4.1.1	Jaccard Similarity	7
4.2	Community detection	8
4.2.1	Louvain	8

1 Exploring and preparing the dataset

1.1 Basic EDA

The dataset we are using is the `Basketball_women.json` dataset. As for any data science project, we begin with an EDA¹

For this, we use Python with the Pandas library, as well as a notebook as it makes prototyping and seeing the result very fast and convenient. We use these tools to get an idea of the structure and the volume of the data.

By looking at the first row, as well as at the result of `df.info()` and the non-empty values for the columns `players_teams` and `awards_players`, we can identify several distinct entities that will constitute the types of the future nodes of our graph. Those are:

¹Exploratory Data Analysis.

```

1 import pandas as pd
2 import numpy as np

1 df = pd.read_json('data/original/Basketball_women.json', lines=True)
2
3 df.iloc[0]

```

id	abrahta0lw
firstName	Tajama
middleName	Ethla
lastName	Abraham
fullGivenName	Tajama Abraham
nameNick	TJ,Taj
pos	C
height	74
weight	190
college	George Washington
birthDate	1975-09-27
birthCity	St.Croix
birthCountry	ISV
highSchool	Kecoughtan
hsCity	Hampton
hsState	VA
hsCountry	USA
players_teams	[{'year': 1997, 'teamID': 'SAC', 'games': 28, 'm...]
awards_players	[]
Name	0, dtype: object

Figure 1: We use Pandas for very basic EDA.

- Player,
- Award,
- College,
- High-School,
- and Team.

Player is the only node with a one-way relationship towards all other node types. In certain cases, as for its relationship with **Team**, the relationship has parameters (e.g. the year, the number of goals, etc).

1.2 Preparing the data

We split the dataset into 5 smaller CSV files. CSV files are easier to import into Neo4j Desktop. Each dataset corresponds to a node type. The Player node type is the only one with properties, all the others only contain an identifier for the specific node. This means we do not need to create datasets for relationships with the Player node type, we can simply package them with the node type.

Those datasets are:

- **awards.csv**: A list of awards, and the relationships between players and awards, as well as relationship information (when was the award won by the player, etc).
- **colleges.csv**
- **high_schools.csv**
- **players_teams.csv**: Similar to awards.csv, but for teams.

- `players.csv`: Lists players information, and their high school and college they attended.

2 Importing the data into Neo4j Desktop

Neo4j Desktop makes it convenient and easy to import data. From its graphical user interface, and after creating an instance and a database for our project, it sufficed to import the CSV files, and determine the different node labels (the name of the type), properties, as well as the relationships.

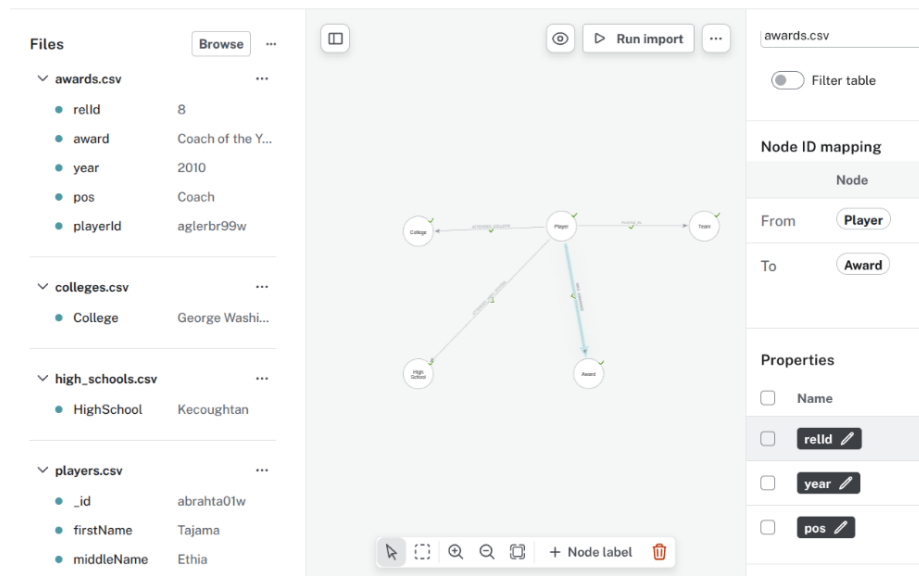


Figure 2: Importing data into Neo4j is intuitive.

2.1 Creating a bi-partite graph

We just created a k -partite graph, because we have several types of nodes and edges only between types. Certain algorithms only run on bi-partite graphs however, meaning graphs with two partitions and with edges only between those partitions.

We create this graph by only considering the "Player" and the "Team" nodes, as they are the relationships with the most information. High school and college are already stored as properties of the "Player" node.

We will create a mono-partite graph later using a projection.

Import results

Total time: 00:00:07

✓

Run import completed

Import completed successfully.

✓

College

colleges.csv

Show Cypher

Time taken	File size	File rows	Nodes created	Properties set	Labels added	Query time
00:00:01	2.0 KiB	160	160	160	160	00:00:01

✓

HighSchool

high_schools.csv

Show Cypher

Time taken	File size	File rows	Nodes created	Properties set	Labels added	Query time
00:00:00	3.1 KiB	241	241	241	241	00:00:00

Figure 3: Importing data is successful.

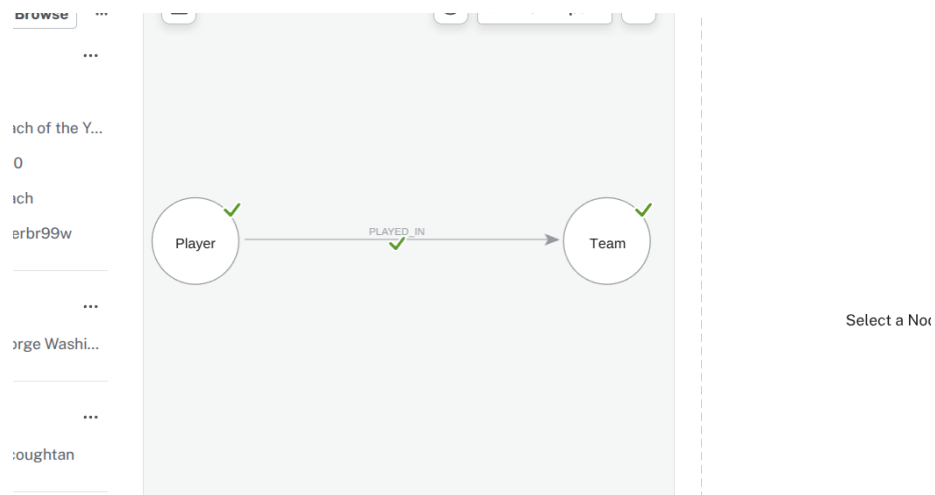


Figure 4: Creating a bipartite graph

3 Exploring the data with Neo4j

With 1878 nodes and relationships, the dataset is huge, and not everything can be seen from the Explore section of Neo4j Desktop.

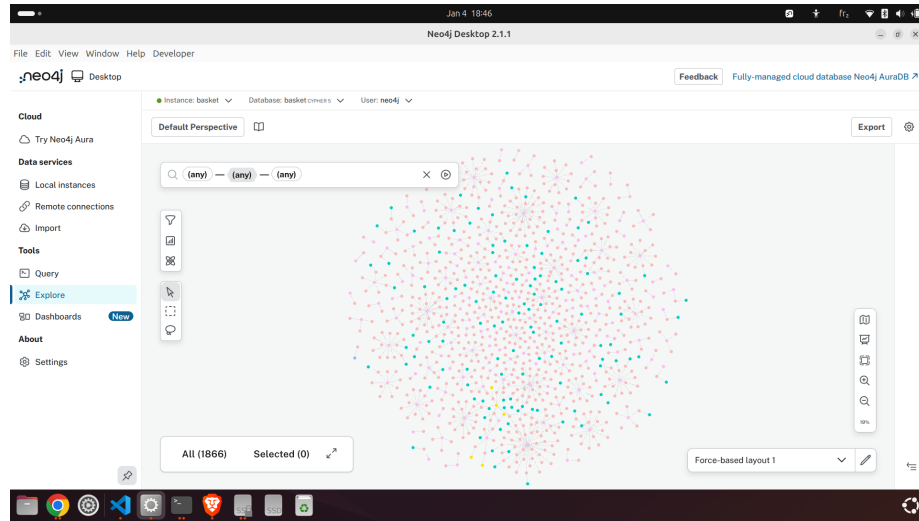


Figure 5: The dataset is huge and not everything can be shown at the same time.

3.1 Creating a projection

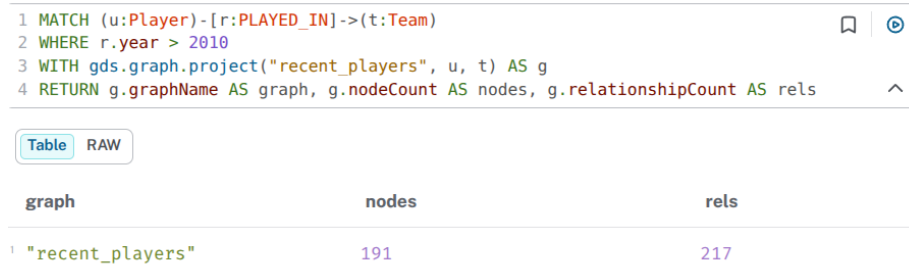
First, we must understand what a **projection** is. This is a very important concept. It can be useful to compare them to **Views** in traditional RDBMS. They are *in-memory* graphs, made from a subset of nodes and relationships of the database graph, and sometimes transformed. As they exist in the DBMS' memory only, they are erased when the instance shuts down. This means they are not part of the database's files.

Projections are not meant to be used like views however. They are not intended to be displayed in Neo4j Desktop's Explorer for instance. They are intended to facilitate the running of algorithms on our data. Indeed, because they are in-memory and not on disk, algorithms can run faster. In addition, projections are decoupled from the database, which prevents accidental modification and makes it possible to create projections with certain characteristics, i.e. only considering players from a certain school for instance.

Now that we understand what a projection is and what purpose they serve, let's create one in Neo4j.

Here, we create a projection based on the result of a **Cypher² query**. This is done by passing the output of the query, which always consists of a table of

²Neo4j's DSL



```

1 MATCH (u:Player)-[r:PLAYED_IN]->(t:Team)
2 WHERE r.year > 2010
3 WITH gds.graph.project("recent_players", u, t) AS g
4 RETURN g.graphName AS graph, g.nodeCount AS nodes, g.relationshipCount AS rels

```

graph	nodes	rels
"recent_players"	191	217

Figure 6: Creating a projection named recent_players

rows even when shown as a graph, to a **Procedure** from the **GDS plugin**. The GDS plugin then creates a projection (hence, "project") from the query's result.

We introduced three important terms: **Procedure**, **Plugin**, and **GDS**.

- A **Procedure** is a function that can modify the database or run advanced algorithms. They are not part of the Cypher DSL. Instead, they are provided by a plugin and can be called in a specific way. (More on that later.)
- A **Plugin** extends the functionalities of Neo4j. It is installed on an instance-basis, meaning a plugin is tied to a specific instance, i.e. a Neo4j server, and not to a database or to a Neo4j Desktop installation.
- **GDS** is an official plugin that provides advanced algorithms for analysing graphs.

For players with a team and who were awarded at least one award:

```

1 MATCH (u:Player)-[r:PLAYED_IN]->(t:Team)
2 WHERE (u)-[:WAS_AWARDED]->(:Award)
3 WITH gds.graph.project("recent_players", u, t) AS g
4 RETURN g.graphName AS graph, g.nodeCount AS nodes, g.
    relationshipCount AS rels

```

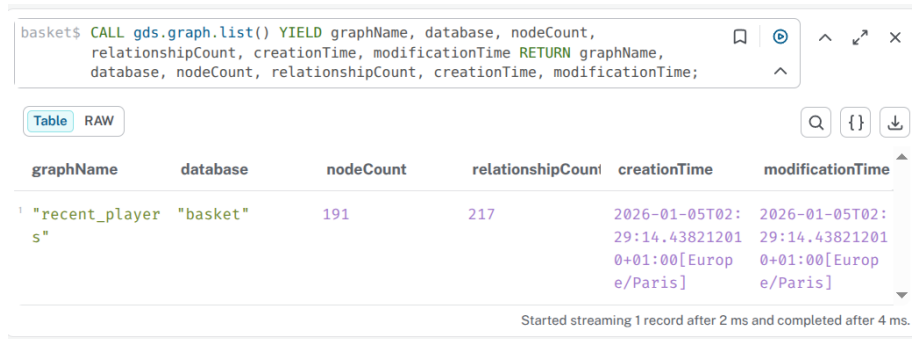
There are simpler manners to create projections, for instance: `CALL gds.graph.project("player_teams",` This creates a projection with all players and all their relationships with teams.

We can then access the list of projections we created. (Figure 7.)

3.1.1 Creating a projection for a monopartite graph

We will now create our monopartite graph as a projection.

To do so, we will only retain Player nodes, and create relationships between them, all that using a projection.



```

basket$ CALL gds.graph.list() YIELD graphName, database, nodeCount,
      relationshipCount, creationTime, modificationTime RETURN graphName,
      database, nodeCount, relationshipCount, creationTime, modificationTime;

```

graphName	database	nodeCount	relationshipCount	creationTime	modificationTime
"recent_players"	"basket"	191	217	2026-01-05T02:29:14.438212010+01:00[Europe/Paris]	2026-01-05T02:29:14.438212010+01:00[Europe/Paris]

Started streaming 1 record after 2 ms and completed after 4 ms.

Figure 7: We can see the list of projections.

```

1  CALL gds.graph.project.cypher(
2  'mono',
3  'MATCH (p:Player) RETURN id(p) AS id',
4  ,
5  MATCH (p1:Player)-[:PLAYED_IN]->(t:Team)<-[:PLAYED_IN]-(p2
    :Player)
6  WHERE id(p1) < id(p2)
7  RETURN id(p1) AS source, id(p2) AS target
8  ,
9  )

```

We get, as a result, 893 nodes (the number of players) with more than 52798 relations in total.

4 Analysis

4.1 Similarity Analysis

A similarity function can be defined as a function that takes two input x and y , and that outputs a score, generally between 0 and 1 (though sometimes between -1 and 1) determining how close the two inputs are.

Several similarity functions are implemented in GDS.

4.1.1 Jaccard Similarity

This similarity functions measures overlap between sets. When applied to a graph such as ours, the sets are the respective nodes' set of neighbours. Meaning that if for two nodes we get a score of 1.0, those two nodes have exactly the same neighbours.

We will apply it to the projection **recent_players** that we created in the previous section, that groups players with an award and a team. As we can see

from the results, certain players share exactly the same team and awards, as indicated by the score of 1. 8

```

1 CALL gds.nodeSimilarity.stream("recent_players", {similarityMetric: "JACCARD"})
2 YIELD node1, node2, similarity
3 RETURN gds.util.asNode(node1).fullGivenName AS U1,
  gds.util.asNode(node2).fullGivenName AS U2, similarity
4 ORDER BY similarity DESCENDING

```

U1	U2	similarity
"Lindsay Whalen"	"Renee Montgomery"	1.0
"Ticha Penicheiro"	"Kristi Toliver"	0.6666666666666666
"Kristi Toliver"	"Ticha Penicheiro"	0.6666666666666666
"Seimone Augustus"	"Lindsay Whalen"	0.5
"Seimone Augustus"	"Renee Montgomery"	0.5

Figure 8: Computing the Jaccard similarity

4.2 Community detection

This allows us to detect communities, meaning group of nodes forming groups within the grap.

One such algorithm is Louvain.

4.2.1 Louvain

```

1 CALL gds.louvain.stream('sct')
2 YIELD nodeId, communityId
3 WITH gds.util.asNode(nodeId) AS n, communityId
4 WHERE 'Player' IN labels(n)
5 RETURN n.fullGivenName AS nodeName, n.highSchool, n.college,
  communityId
6 ORDER BY communityId

```

Notice this part WHERE 'Player' IN labels(n), whic filters out non-Player nodes.

This algorithm allows us to detect communities inside our graph. (Figure 9)

What is unexpected from the results we got is that players rarely seem to come from the same College. The algorithm might be lead into thinking that they share the same high school, simply because the high school is set to null. Maybe only considering players with a high school filled and a college will allow the algorithm to run smoother.

Table RAW			
nodeName	n.highSchool	n.college	communityId
¹ "Monique Ambers"	null	"Arizona State"	47
² "Chantelle Anderson"	null	"Vanderbilt"	47
³ "Marlies Askamp"	null	"none"	47
⁴ "Morenike Atunrase"	null	"Texas A&M"	47
⁵ "Rhonda Smith Banchero"	null	"Washington"	47
⁶ "LaQuanda Barksdale"	null	"North Carolina"	47
⁷ "Adia Barnes"	"Mission Bay"	"Arizona"	47
⁸ "Quacy Barnes"	"Benton Harbor"	"Indiana"	47

Figure 9: Detecting communities

This is because the Louvain algorithm expects a **monopartite** graph. Let's try again with one such graph.

As we can see in figure 10, the communities that were identified now make more sense. They reveal what teams the players played in, as this is based on the graph's relationships (which in our mono-partite projection only reveals the teams the players played in).

Future work may include creating new relationships between Players (for instance, if they went to the same high school or college), and run Louvain specifically on these relationships or on all relationships together, to see the impact it has on communities. However, Louvain is not deterministic. Change will happen every time it is run.

6 ORDER BY communityId

Table RAW

	nodeName	n.highSchool	n.college	communityId
94	"Sharon Thompson"	null	null	775
95	"Heidi VanDerveer"	null	null	799
96	"Boky Vidic"	null	"Oregon State"	806
97	"Lauretta Freeman"	null	"Auburn"	806
98	"Cornelia Gayden"	null	"Louisiana State"	806
99	"Rachel Hemmer"	null	"Stanford"	806
100	"Tracy Henderson"	"Patrick Henry"	"Georgia"	806
101	"Amy Chambers"	null	"Yale"	806

Figure 10: Running Louvain on the mono-partite projection.